

人工智能所需数学基础

数学是A I 的语言。不理解数学，你只能调用A P I ；理解了数学，你才能创造算法。本文将系统化地梳理A I 核心数学知识，并深入到推导和直觉层面。

人工智能所需数学基础

数学是A I 的语言。不理解数学，你只能调用A P I ；理解了数学，你才能创造算法。本文将系统化地梳理A I 核心数学知识，并深入到推导和直觉层面。

- - -

一、线性代数：A I 的基础语言

线性代数是A I 技术的基础之一，它为A I 算法提供了数据表示和运算的基本工具。无论是神经网络、支持向量机，还是主成分分析（P C A），线性代数的概念如向量、矩阵、线性变换等都贯穿其中。

1.1 向量空间和子空间

向量空间的正式定义：

一个向量空间 V

是一个集合，其元素称为向量，定义了两种运算（加法和标量乘法），满足以下八条公理（对任意 $\mathbf{u}, \mathbf{v}, \mathbf{w} \in V$ 和标量 a ）

公理 加法性质 标量乘法性质

封闭性 $\mathbf{u} + \mathbf{v} \in V$ $a\mathbf{u} \in V$

交换律 $\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}$

结合律 $(\mathbf{u} + \mathbf{v}) + \mathbf{w} = \mathbf{u} + (\mathbf{v} + \mathbf{w})$
 $a(b\mathbf{u}) = (ab)\mathbf{u}$

分配律 — $a(\mathbf{u} + \mathbf{v}) = a\mathbf{u} + a\mathbf{v}$

单位元 存在 $\mathbf{0}$ 使得 $\mathbf{u} + \mathbf{0} = \mathbf{u}$

逆元 存在 $-\mathbf{u}$ 使得 $\mathbf{u} + (-\mathbf{u}) = \mathbf{0}$

为什么向量空间对A I 重要？

在A I 中，所有数据最终都表示为向量空间中的点：

- 一张 28×28 的灰度图像 $\rightarrow \mathbb{R}^{784}$ 中的一个点
- 一个100维的词向量 $\rightarrow \mathbb{R}^{100}$ 中的一个点

- 一个神经网络的所有参数 $\rightarrow$ 某个高维向量空间中的一个点

子空间：

如果 W 是 V 的子集, 且 W 本身也是向量空间, 则 W 是 V 的子空间。

判断子空间的充要条件： W 是 V 的子空间, 当且仅当：

1. $\mathbf{0} \in W$
2. 若 $\mathbf{u}, \mathbf{v} \in W$, 则 $\mathbf{u} + \mathbf{v} \in W$
3. 若 $\mathbf{u} \in W$, c 为标量, 则 $c\mathbf{u} \in W$

AI 中的子空间示例：

```
import numpy as np
```

在人脸识别中, "人脸空间" 是高维图像空间的子空间

所有 80×80 灰度人脸图像构成一个约 R^{6400} 的向量

但实际上人脸只占据该空间的一个低维子空间 (约 $50 - 200$ 维)

用 PCA 提取人脸子空间 (Eigenfaces 方法)

```
def compute_eigenfaces(faces, ncomponents=100):
    """
    faces: (nsamples, npixels) 每行是一张展平的人脸图像
    返回: 前ncomponents个主成分(特征脸)
    """
    # 中心化
    mean_face = faces.mean(axis=0)
    centered = faces - mean_face

    # 计算协方差矩阵
    # 如果样本数 < 像素数, 使用 "技巧": 计算  $X^T X$  而非  $X X^T$ 
    covmatrix = centered @ centered.T # (nsamples, nsamples)

    # 特征分解
    eigenvalues, eigenvectors = np.linalg.eigh(covmatrix)

    # 转换回原始空间
```

```

eigenfaces = centered.T @ eigenvectors # (npixels

# 选取前ncomponents个
eigenfaces = eigenfaces[:, -ncomponents:]

# 归一化
eigenfaces = eigenfaces / np.linalg.norm(eigenface

return eigenfaces, meanface

```

人脸可以表示为特征脸的线性组合

这意味着所有可能的 " 人脸 " 都存在于一个低维子空间中

新人脸 = meanface + w1eigenface1 + w2eigenface2 + . . .

线性组合与张成空间：

向量 $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k$ 的所有线性组合构成的集合称为它们的张成空间 (Span)：

$$\text{Span}(\mathbf{v}_1, \dots, \mathbf{v}_k) = \{ \sum_{i=1}^k c_i \mathbf{v}_i \mid c_i \in \mathbb{R} \}$$

AI 直觉：神经网络的每一层将输入映射到一个新的张成空间。全连接层的权重矩阵的列向量定义了该层能表示的所有可能的输出方向——这就是为什么网络宽度（神经元数量）决定了表达能力。

1.2 线性变换的几何意义

线性变换的正式定义：

映射 $T: V \rightarrow W$ 是线性变换，当且仅当：

- $T(\mathbf{u} + \mathbf{v}) = T(\mathbf{u}) + T(\mathbf{v})$
- $T(c\mathbf{u}) = cT(\mathbf{u})$ (保持标量乘法)

等价表述： $T(a\mathbf{u} + b\mathbf{v}) = aT(\mathbf{u}) + bT(\mathbf{v})$

几何直觉：

线性变换对空间的几何效果只有四种基本操作：

操作 矩阵示例 几何效果

旋转 $\begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$
保持长度和角度，旋转空间

缩放 $\begin{pmatrix} s_x & 0 \\ 0 & s_y \end{pmatrix}$

剪切 $\begin{pmatrix} 1 & k \\ 0 & 1 \end{pmatrix}$ 平行

反射 $\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$

关键性质：

- 线性变换保持原点不动 ($T(\mathbf{0}) = \mathbf{0}$)
- 线性变换保持直线 (直线变换后仍是直线)
- 线性变换保持平行线平行
- 任何线性变换都可以用矩阵乘法表示

```
import numpy as np
import matplotlib.pyplot as plt
```

可视化线性变换对单位正方形的几何效果

```
def visualize_transform(matrix, title):
    """展示线性变换如何改变空间"""
    # 单位正方形的四个顶点
    square = np.array([[0, 0], [1, 0], [1, 1], [0, 1]])

    # 应用变换
    transformed = matrix @ square

    # 原始单位圆上的点
    theta = np.linspace(0, 2*np.pi, 100)
    circle = np.array([np.cos(theta), np.sin(theta)])
    transformed_circle = matrix @ circle

    # 绘制
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 6))

    ax1.plot(square[0], square[1], 'b-')
    ax1.plot(circle[0], circle[1], 'r-')
    ax1.set_title('变换前')
    ax1.set_xlim(-3, 3); ax1.set_ylim(-3, 3)
```

```

ax1.grid(True); ax1.setaspect('equal')

ax2.plot(transformed[0], transformed[1], 'b-')
ax2.plot(transformedcircle[0], transformedcircle[1], 'b-')
ax2.settitle(f'变换后: {title}')
ax2.setxlim(-3, 3); ax2.setylim(-3, 3)
ax2.grid(True); ax2.setaspect('equal')

plt.show()

```

示例：旋转变换

```

rotation = np.array([[np.cos(np.pi/4), -np.sin(np.pi/4)],
                      [np.sin(np.pi/4), np.cos(np.pi/4)]])
visualizetransform(rotation, '旋转45°')

```

示例：剪切变换

```

shear = np.array([[1, 1], [0, 1]])
visualizetransform(shear, '剪切')

```

神经网络中的线性变换：

神经网络的每一层就是一个线性变换 + 非线性激活

$$y = \sigma(Wx + b)$$

Wx 是线性变换：将输入从输入空间映射到输出空间

σ 是非线性激活：打破线性性，使网络能学习复杂模式

b 是偏置：平移变换（注意：有偏置的变换是仿射变换，不是严格的线性变换）

为什么需要非线性激活？

如果所有层都是线性的，那么多层网络等价于一个单层网络！

证明：设两层线性网络为 $y = W_2(W_1x + b_1) + b_2 = W_2W_1x + (W_2b_1 + b_2)$

令 $W = W_2W_1$, $b = W_2b_1 + b_2$, 则 $y = Wx + b$

两层线性网络 $\equiv$ 一层线性网络（没有增加表达能力！）

1.3 向量和矩阵的表示

在AI中，数据通常被表示为向量或矩阵形式：

- 图像可以表示为一个高维向量

- 文本可以表示为词向量
- 时间序列数据可以表示为矩阵

```
import numpy as np
```

向量运算

```
vectora = np.array([1, 2, 3])
vectorb = np.array([4, 5, 6])
vectoradd = vectora + vectorb # 向量加法
```

矩阵运算

```
matrixa = np.array([[1, 2], [3, 4]])
matrixb = np.array([[5, 6], [7, 8]])
matrixmul = np.dot(matrixa, matrixb) # 矩阵乘法
```

1.4 核心概念

概念 说明 AI应用

向量 具有大小和方向的量 词向量、特征向量

矩阵 二维数值数组 权重矩阵、数据矩阵

张量 多维数组 深度学习核心数据结构

矩阵乘法 线性变换的核心运算 Transformer注意力机制

特征值 / 特征向量 矩阵本征性质 PCA降维、图谱分析

奇异值分解 (SVD) 矩阵分解方法 推荐系统、数据压缩

范数 向量 / 矩阵的大小度量 正则化、梯度裁剪

1.5 矩阵分解详解

矩阵分解是线性代数中最重要工具之一，在AI中有广泛的应用。

1.5.1 特征分解 (Eigendecomposition)

定义：对于方阵 A ，如果存在非零向量 $\mathbf{v}$ 和标量 λ 使得

$$A\mathbf{v} = \lambda\mathbf{v}$$

则 λ 称为特征值， $\mathbf{v}$ 称为特征向量。

几何意义：特征向量是矩阵变换后方向不变的向量，特征值表示在该方向上的缩放比例。

变换前： 变换后（A变换）：

↑ v_2 ↗ v_2 （方向改变，不是特征向量）

/

↗ v_1 | —————→ v_1' （方向不变！是特征向量，长度变为 λ 倍）

/ |

└──────────┬──────────┘

v_1 是特征向量 $v_1' = \lambda \cdot v_1$

特征分解定理：如果矩阵 A 有 n 个线性无关的特征向量，则可以分解为：

$$A = Q \Lambda Q^{-1}$$

其中 Q 是特征向量组成的矩阵， Λ 是特征值组成的对角矩阵。

```
import numpy as np
```

特征分解示例

```
A = np.array([[4, 2], [1, 3]])
```

```
eigenvalues, eigenvectors = np.linalg.eig(A)
```

```
print(f"特征值: {eigenvalues}") # [5. 2.]
```

```
print(f"特征向量: \n{eigenvectors}")
```

验证： $Av = \lambda v$

```
for i in range(len(eigenvalues)):
```

```
    v = eigenvectors[:, i]
```

```
    lambda_i = eigenvalues[i]
```

```
    print(f"A v = {(A @ v).round(4)}, λ v = {(lambda_i * v).round(4)}")
```

重构原始矩阵

```
Q = eigenvectors
```

```
Lambda = np.diag(eigenvalues)
```

```
Areconstructed = Q @ Lambda @ np.linalg.inv(Q)
```

```
print(f"重构矩阵: \n{Areconstructed.round(4)}") # 应该等于A
```

1.5.2 SVD (奇异值分解)

SVD定理：任何实矩阵 $A \in \mathbb{R}^{m \times n}$ 都可以分解

$$A = U \Sigma V^T$$

其中：

- $U \in \mathbb{R}^{m \times m}$ ：左奇异向量矩阵（正交矩阵）
- $\Sigma \in \mathbb{R}^{m \times n}$ ：奇异值矩阵（对角矩阵）
- $V \in \mathbb{R}^{n \times n}$ ：右奇异向量矩阵（正交矩阵）

SVD与特征分解的关系：

$$AA^T = U \Sigma V^T V \Sigma^T U^T = U \Sigma \Sigma^T U^T$$

$$A^T A = V \Sigma^T U^T U \Sigma V^T = V \Sigma^T \Sigma V^T$$

所以： U 是 AA^T 的特征向量， V 是 $A^T A$ 的特征向量，奇异值是 AA^T （或 $A^T A$ ）特征值的平方根。

SVD的几何意义：

任何线性变换都可以分解为三个步骤：

1. V^T ：在输入空间中旋转
2. Σ ：沿坐标轴缩放（奇异值就是缩放比例）
3. U ：在输出空间中旋转

```
import numpy as np
```

SVD示例

```
A = np.array([[3, 2, 2],  
[2, 3, -2]])
```

```
U, sigma, VT = np.linalg.svd(A, fullmatrices=False)
```

```
print(f"U (左奇异向量): \n{U.round(4)}")
```

```
print(f"奇异值: {sigma.round(4)}")
```

```
print(f"V^T (右奇异向量): \n{VT.round(4)}")
```


验证重构

```
Areconstructed = U @ np.diag(sigma) @ VT
print(f"重构矩阵:\n{Areconstructed.round(4)}")
```

SVD在推荐系统中的应用——矩阵补全：

```
import numpy as np
```

用户 - 电影评分矩阵（0 表示未评分）

```
R = np.array([
[5, 3, 0, 1], # 用户1的评分
[4, 0, 0, 1], # 用户2
[1, 1, 0, 5], # 用户3
[1, 0, 0, 4], # 用户4
[0, 1, 5, 4], # 用户5
])
```

使用截断SVD进行矩阵补全

```
def recommendsvd(R, k=2):
    """
    R: 评分矩阵 (nusers, nitems)
    k: 保留的奇异值个数（隐因子维度）
    """
    # 步骤1：填充缺失值为均值
    mask = R > 0
    meanrating = R[mask].mean()
    Rfilled = R.copy()
    Rfilled[~mask] = meanrating

    # 步骤2：SVD分解
    U, sigma, VT = np.linalg.svd(Rfilled, fullmatrices=True)

    # 步骤3：截断——只保留前k个奇异值
    Uk = U[:, :k] # (nusers, k)
    sigma_k = sigma[:k] # (k,)
```

```

V T k  =  V T [ : k ,  : ]  #  ( k ,  n i t e m s )

# 步骤4：重构评分矩阵
R p r e d  =  U k  @  n p . d i a g ( s i g m a k )  @  V T k

# 步骤5：推荐——对未评分项预测评分最高的
R p r e d [ m a s k ]  =  - n p . i n f  # 排除已评分的项
r e c o m m e n d a t i o n s  =  n p . a r g m a x ( R p r e d ,  a x i s = 1 )

r e t u r n  R p r e d ,  r e c o m m e n d a t i o n s

R p r e d ,  r e c s  =  r e c o m m e n d s v d ( R ,  k = 2 )
p r i n t ( f " 预测评分矩阵：\n{ R p r e d . r o u n d ( 2 ) } " )
p r i n t ( f " 每人的推荐项： { r e c s } " )

```

直觉解释：

U_k 的每一行代表用户在 k 维隐因子空间中的位置

$V T_k$ 的每一列代表物品在 k 维隐因子空间中的位置

它们的乘积就是 " 用户偏好 " 和 " 物品特征 " 的匹配程度

例如： $k = 2$ 时，隐因子可能对应 " 动作片偏好 " 和 " 文艺片偏好 "

SVD在PCA中的应用：

```

import numpy as np
from sklearn.datasets import load_iris

```

加载数据

```

i r i s  =  l o a d _ i r i s ( )
X  =  i r i s . d a t a  #  ( 1 5 0 ,  4 )
y  =  i r i s . t a r g e t

```

PCA using SVD

```

def pcasvd ( X ,  n c o m p o n e n t s = 2 ) :
    " " "

```

使用SVD实现PCA

比先计算协方差矩阵再特征分解更数值稳定

```

    " " "

```

```

# 中心化
Xmean = X.mean(axis=0)
Xcentered = X - Xmean

# SVD分解
# 注意：对数据矩阵直接做SVD，而非对协方差矩阵做特征分解
# 这在数值上更稳定（特别是当维度 >> 样本数时）
U, sigma, VT = np.linalg.svd(Xcentered, fullmatrix=True)

# 主成分 = V的前k列（VT的前k行转置）
components = VT[:ncomponents] # (ncomponents, nfeatures)

# 投影到主成分空间
Xtransformed = Xcentered @ components.T # (nsamples, ncomponents)

# 解释方差比例
totalvar = np.sum(sigma**2) / (len(X) - 1)
explainedvar = (sigma[:ncomponents]**2) / (len(X) - 1)
explainedratio = explainedvar / totalvar

return Xtransformed, components, explainedratio

Xpca, components, explainedratio = pcasvd(X, ncomponents)
print(f"解释方差比例： {explainedratio.round(4)}") # 约 0.85
print(f"前2个主成分解释了 {explainedratio.sum():.2%} 的方差")

```

1.5.3 LU分解

$$PA = LU$$

其中 P 是置换矩阵， L 是下三角矩阵， U 是上三角矩阵。

应用：高效求解线性方程组 $Ax = b$ ：

1. 分解 $PA = LU$
2. 解 $Ly = Pb$ （前向替换）
3. 解 $Ux = y$ （后向替换）

```

import numpy as np
from scipy.linalg import lu

A = np.array([[2, 1, 1],
[4, 3, 3],
[8, 7, 9]])

P, L, U = lu(A)
print(f"P: \n{P}")
print(f"L: \n{L.round(4)}")
print(f"U: \n{U.round(4)}")
print(f"验证 PA = LU: \n{(P @ A).round(4)} \n{(L @ U).round(4)}")

```

1.5.4 QR分解

$$A = QR$$

其中 Q 是正交矩阵， R 是上三角矩阵。

应用：

1. 求解最小二乘问题（比正规方程更数值稳定）
2. 计算特征值的QR算法
3. Gram-Schmidt正交化

```

import numpy as np

A = np.array([[1, 1, 0],
[1, 0, 1],
[0, 1, 1]])

Q, R = np.linalg.qr(A)
print(f"Q (正交矩阵): \n{Q.round(4)}")
print(f"R (上三角矩阵): \n{R.round(4)}")
print(f"验证 Q^T Q = I: \n{(Q.T @ Q).round(4)}") # 应该接近单位矩阵

```

1.6 张量运算详解

张量的定义：张量是向量和矩阵的推广。0阶张量是标量，1阶张量是向量，2阶张量是矩阵，n阶张量是n维张量。

阶数 名称 形状示例 AI应用

0 标量 () 损失值、学习率

1 向量 (7 6 8 ,) 词向量、偏置

2 矩阵 (7 6 8 , 7 6 8) 权重矩阵

3 3阶张量 (batch , seq len , hidden) 批量序列数据

4 4阶张量 (batch , channels , height , width) 批量图像数据

```
import torch
```

张量运算在深度学习中的核心操作

1. 张量创建

```
x = torch.randn(32, 128, 768) # (batchsize, seq len, hidden)
print(f"形状: {x.shape}")
```

2. 张量重塑 (reshape / view)

```
x2d = x.reshape(32 * 128, 768) # 合并前两个维度
```

```
x3d = x2d.reshape(32, 128, 768) # 恢复
```

```
xpermuted = x.permute(1, 0, 2) # 交换前两个维度: (128, 32, 768)
```

3. 矩阵乘法 (核心运算)

```
W = torch.randn(768, 768) # 权重矩阵
```

```
output = x @ W # (32, 128, 768) — 广播的矩阵乘法
```

4. Einstein求和约定 (einsum) ——灵活的张量运算

自注意力中的 QK^T 计算

```
Q = torch.randn(32, 8, 128, 96) # (batch, heads, seq len, head dim)
```

```
K = torch.randn(32, 8, 128, 96) # (batch, heads, seq len, head dim)
```

使用einsum进行批量矩阵乘法

```
attnscores = torch.einsum('bhqd, bhkd -> bhqk', Q, K)
```

'bhqd, bhkd -> bhqk' 含义:

两个输入的 'bh' 维度对应相乘求和 (batch和heads维度广播)

'q' 和 'k' 维度保留在输出中

'd' 维度求和消失 (点积)

```
print(f"注意力分数形状: {attnscores.shape}") # (32, 8, 1
```

5. 张量广播

```
bias = torch.randn(768) # (768,)
```

```
result = x + bias # (32, 128, 768) + (768,) → 自动广播
```

广播规则:

1. 从最右侧维度开始对齐

2. 每个维度必须相等, 或其中一方为 1, 或一方不存在

$(32, 128, 768) + (768,) \rightarrow (768,) \rightarrow (1, 1, 768) \rightarrow$

张量与深度学习的联系:

现代深度学习框架 (PyTorch、TensorFlow) 的核心就是张量计算。GPU 之所以适合深度学习为高效处理大规模张量运算:

一个 Transformer 层的张量运算流程

```
class TransformerLayer:
```

```
def forward(self, x):
```

```
# x: (batch, seq_len, d_model)
```

```
# 1. 线性变换 (张量乘法)
```

```
Q = x @ self.Wq # (batch, seq_len, d_model) @ (d_model, d_k)
```

```
K = x @ self.Wk
```

```
V = x @ self.Wv
```

```
# 2. 注意力计算 (张量缩并)
```

```
scores = Q @ K.transpose(-2, -1) / math.sqrt(d_k)
```

```
attn = softmax(scores, dim=-1)
```

```
# 3. 加权求和
```

```
context = attn @ V # 批量矩阵乘法
```

```
# 4. 输出变换
```

```
output = context @ self.Wo
```

整个前向传播就是一系列张量运算的组合！

return output

- - -

二、概率论：AI 的决策基础

概率论是 AI 技术中的另一个重要支柱。无论是机器学习中的分类算法，还是强化学习中的决策过程，概率论都提供了理论基础。

2.1 概率的公理化定义（Kolmogorov 公理）

概率空间由三元组 $(\Omega, \mathcal{F}, P)$ 定义：

- Ω ：样本空间，所有可能结果的集合
- $\mathcal{F}$ ：事件空间， Ω 的子集的 σ -代数（包含空集和全集）
- P ：概率测度，满足以下三条公理：

Kolmogorov 三条公理：

公理 数学表述 直觉含义

非负性 $P(A) \geq 0$ ，对任意 $A \in \mathcal{F}$ 概率不可能是负数

规范性 $P(\Omega) = 1$ 所有结果中必有某个发生

可列可加性 若 $A_1, A_2, \dots$ 互斥，则 $P(\bigcup_{i=1}^{\infty} A_i) = \sum_{i=1}^{\infty} P(A_i)$ 互斥事件的概率可以相加

从这三条公理可以推出所有概率规则，例如：

- $P(\emptyset) = 0$ （不可能事件的概率为 0）
- $P(A^c) = 1 - P(A)$ （对立事件的概率互补）
- $P(A \cup B) = P(A) + P(B) - P(A \cap B)$ （容斥原理）

2.2 条件概率与全概率公式

条件概率定义：

$$P(A|B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0$$

直觉：在已知 B 发生的条件下， A 发生的概率。条件概率将样本空间从 Ω 缩小到 B 。

全概率公式：

如果 $B_1, B_2, \dots, B_n$ 是样本空间的一个划分（互斥且并集为 Ω ）

$$P(A) = \sum_{i=1}^n P(A \cap B_i) = \sum_{i=1}^n P(A|B_i)P(B_i)$$

推导：

$$P(A) = P(A \cap \Omega) = P\left(A \cap \bigcup_{i=1}^n B_i\right) = \sum_{i=1}^n P\left(A \cap B_i\right)$$

因为 B_i 互斥，所以 $(A \cap B_i)$ 也互斥：

$$P(A) = \sum_{i=1}^n P(A \cap B_i) = \sum_{i=1}^n P(A|B_i)P(B_i)$$

2.3 贝叶斯定理：详细推导与应用

贝叶斯定理：

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

推导：

由条件概率定义：

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \quad \text{即} \quad P(A \cap B) = P(A|B)P(B)$$

同样：

$$P(B|A) = \frac{P(A \cap B)}{P(A)} \quad \text{即} \quad P(A \cap B) = P(B|A)P(A)$$

从 (2) 得到： $P(A \cap B) = P(B|A)P(A)$

代入 (1)：

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

贝叶斯定理的深刻含义：

$$\underbrace{P(A|B)}_{\text{后验概率}} = \frac{\underbrace{P(B|A)}_{\text{似然}} \underbrace{P(A)}_{\text{先验概率}}}{\underbrace{P(B)}_{\text{证据}}}$$

术语 含义 AI 中的角色

先验概率 $P(A)$ 观测前的初始信念 模型的初始假设、正则化项

似然度 $P(B \mid A)$ 在假设 A 下观测到 B 的可能性 模型对数据的拟合程度

证据 $P(B)$ 观测到的数据的总概率 归一化常数（通常难以计算）

后验概率 $P(A \mid B)$ 观测后更新的信念 模型的最终推断结果

贝叶斯定理在垃圾邮件分类中的应用：

```
import numpy as np
from collections import defaultdict

class NaiveBayesSpamFilter:
    """ 朴素贝叶斯垃圾邮件分类器 """

    def __init__(self):
        self.spamwordcounts = defaultdict(int)
        self.hamwordcounts = defaultdict(int)
        self.spamttotalwords = 0
        self.hamtotalwords = 0
        self.nspam = 0
        self.nham = 0
        self.vocab = set()

    def train(self, emails):
        """
        emails: [(text, label), ...] label: 'spam' or 'ham'
        """
        for text, label in emails:
            words = text.lower().split()

            if label == 'spam':
                self.nspam += 1
                for word in words:
                    self.spamwordcounts[word] += 1
```

```

self.spamttotalwords += 1
else:
self.nham += 1
for word in words:
self.hamwordcounts[word] += 1
self.hamttotalwords += 1

self.vocab.update(words)

```

```

def predict(self, text):
"""

```

计算 $P(\text{spam text})$ 和 $P(\text{ham text})$ ，取较大者

```

"""

```

```

words = text.lower().split()
vocabsize = len(self.vocab)

```

先验概率

```

totalemails = self.nspam + self.nham
logpspam = np.log(self.nspam / totalemails)
logpham = np.log(self.nham / totalemails)

```

似然度（使用拉普拉斯平滑避免零概率）

alpha = 1 # 平滑参数

```

for word in words:

```

```

#  $P(\text{word spam}) = (\text{count}(\text{word in spam}) + \alpha) /$ 
pwordspam = (self.spamwordcounts[word] + alpha) / \
(self.spamttotalwords + alpha vocabsize)
pwordham = (self.hamwordcounts[word] + alpha) / \
(self.hamttotalwords + alpha vocabsize)

```

```

logpspam += np.log(pwordspam)

```

```

logpham += np.log(pwordham)

```

```

return 'spam' if logpspam > logpham else 'ham'

```

贝叶斯推断过程：

```
# P ( spam email ) ∝ P ( email spam ) × P ( spam )
# = P ( w1 spam ) × P ( w2 spam ) × . . . × P ( spam ) [ 朴素假设 ]
# 取对数避免数值下溢：
# log P ( spam email ) ∝ log P ( spam ) + Σ log P ( wi sp
```

使用示例

```
emails = [
    ( "buy cheap viagra now", "spam" ),
    ( "meeting at 3pm today", "ham" ),
    ( "win free prize click here", "spam" ),
    ( "project deadline extended", "ham" ),
    ( "congratulations you won lottery", "spam" ),
    ( "lunch at the cafeteria", "ham" ),
]
```

```
filter = NaiveBayesSpamFilter()
filter.train(emails)
```

```
testemail = "you won free viagra"
```

```
print( f" '{testemail}' is {filter.predict(testemail)}
```

输出： 'spam' — 因为 "free" 和 "viagra" 在spam中概率高

2.4 常见概率分布及其在AI中的应用

基础分布

分布 公式 / 特点 AI应用

高斯分布 $f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp(-\frac{x^2}{2\sigma^2})$ 建模连续数据、GAN噪声、VAE隐变量、权重初始化

伯努利分布 $P(X=1)=p$, $P(X=0)=1-p$ 二分类 (逻辑回归输出层)

多项式分布 $P(X=i) = p_i$ 多分类 (softmax输出层)

泊松分布 $P(X=k) = \frac{\lambda^k e^{-\lambda}}{k!}$

Beta分布——贝叶斯推断的共轭先验

概率密度函数：

$$f(x; \alpha, \beta) = \frac{x^{\alpha-1} (1-x)^{\beta-1}}{B(\alpha, \beta)}$$

其中 $B(\alpha, \beta) = \frac{\Gamma(\alpha)\Gamma(\beta)}{\Gamma(\alpha+\beta)}$ 是Beta函数。

关键性质：

- 定义域为 $[0, 1]$ ——天然适合建模概率
- α 和 β 控制分布形状：
- $\alpha > \beta$ ：偏向1
- $\alpha < \beta$ ：偏向0
- $\alpha = \beta$ ：对称
- $\alpha = \beta = 1$ ：均匀分布

AI 应用——在线学习的贝叶斯更新：

```
import numpy as np
from scipy import stats
```

```
class BayesianABTest:
    """
```

使用Beta分布作为共轭先验进行A / B测试

场景：两个网页版本，哪个点击率更高？

```
"""
```

```
def __init__(self):
```

```
# 先验：Beta(1, 1) = 均匀分布（无信息先验）
```

```
self.alphaA, self.betaA = 1, 1 # 版本A的参数
```

```
self.alphaB, self.betaB = 1, 1 # 版本B的参数
```

```
def update(self, variant, clicked):
```

```
    """
```

观测到一次点击 / 未点击事件后更新后验

Beta分布是伯努利似然的共轭先验：

$P(\theta | \text{data}) = \text{Beta}(\alpha + \text{点击数}, \beta + \text{未点击数})$

```
    """
```

```
    if variant == 'A':
```

```

if clicked:
    self.alphaA += 1
else:
    self.betaA += 1
else:
    if clicked:
        self.alphaB += 1
    else:
        self.betaB += 1

def probabilityBbetter(self, nsamples=100000):
    """ 估计版本B优于版本A的概率 """
    samplesA = stats.beta.rvs(self.alphaA, self.betaA)
    samplesB = stats.beta.rvs(self.alphaB, self.betaB)
    return (samplesB > samplesA).mean()

def expectedclickrate(self, variant):
    """ 预测点击率的期望值 """
    if variant == 'A':
        return self.alphaA / (self.alphaA + self.betaA)
    else:
        return self.alphaB / (self.alphaB + self.betaB)

```

使用示例

```
test = BayesianABTest()
```

模拟数据：版本B实际上有更高的点击率

```

np.random.seed(42)
for i in range(100):
    test.update('A', np.random.random() < 0.10) # A:
    test.update('B', np.random.random() < 0.15) # B:

print(f"P(B > A) = {test.probabilityBbetter():.2%}")
print(f"预期点击率 - A: {test.expectedclickrate('A'):.2%}")
print(f"预期点击率 - B: {test.expectedclickrate('B'):.2%}")

```

Dirichlet 分布——多项式分布的共轭先验

概率密度函数：

$$f(\mathbf{x}; \boldsymbol{\alpha}) = \frac{1}{Z} \prod_{i=1}^K x_i^{\alpha_i - 1}$$

其中 $\mathbf{x}$ 在单纯形上 ($x_i \geq 0$, $\sum x_i = 1$)
 $\dots, \alpha_K$)。

AI 应用——LDA 主题模型：

LDA (Latent Dirichlet Allocation) 的概率图模型

文档生成过程：

1. 对每个文档 d ：

a. 从 $\text{Dirichlet}(\boldsymbol{\alpha})$ 抽取主题分布 $\boldsymbol{\theta}_d \sim \text{Dir}(\boldsymbol{\alpha})$

b. 对每个词位 n ：

i. 从 $\text{Multinomial}(\boldsymbol{\theta}_d)$ 抽取主题 $z_{dn} \sim \text{Mult}(\boldsymbol{\theta}_d)$

ii. 从 $\text{Multinomial}(\boldsymbol{\phi}_{z_{dn}})$ 抽取词 $w_{dn} \sim \text{Mult}(\boldsymbol{\phi}_{z_{dn}})$

2. 对每个主题 k ：

a. 从 $\text{Dirichlet}(\boldsymbol{\beta})$ 抽取词分布 $\boldsymbol{\phi}_k \sim \text{Dir}(\boldsymbol{\beta})$

Dirichlet 分布的参数 $\boldsymbol{\alpha}$ 控制主题分布的 "集中度"：

$\boldsymbol{\alpha}$ 大 $\rightarrow$ 文档倾向于均匀覆盖所有主题

$\boldsymbol{\alpha}$ 小 $\rightarrow$ 文档倾向于只涉及少数主题

这种层次化的贝叶斯模型是主题建模的基础

指数族分布

统一形式：

$$p(\mathbf{x}; \boldsymbol{\eta}) = \frac{h(\mathbf{x}) \exp(\boldsymbol{\eta}^T \mathbf{T}(\mathbf{x}))}{A(\boldsymbol{\eta})}$$

其中：

- $\boldsymbol{\eta}$ ：自然参数

- $\mathbf{T}(\mathbf{x})$ ：充分统计量

- $h(x)$: 基底测度
- $A(\boldsymbol{\eta})$: 对数配分函数 (归一化常数)

分布 自然参数 $\boldsymbol{\eta}$ 充分统计量 $T(x)$ AI 应用

伯努利 $\log \frac{p}{1-p}$ x 二分类

高斯 (已知方差) μ / σ^2 x , x^2 回归

泊松 $\log \lambda$ x 计数数据

指数 $-\lambda$ x 等待时间

Dirichlet $\alpha_i - 1$ $\log x_i$ 主题模型

为什么指数族重要?

许多 AI 算法的推导在指数族框架下是统一的。例如, 广义线性模型 (GLM)、变分推断、指数族 PCA 等。

2.5 极大似然估计与极大后验估计

极大似然估计 (MLE)

目标: 找到使观测数据出现概率最大的参数值。

$$\hat{\boldsymbol{\theta}}_{\text{MLE}} = \arg \max_{\boldsymbol{\theta}} \prod_{i=1}^n P(\mathbf{x}_i | \boldsymbol{\theta})$$

取对数 (化积为和, 且不改变最大值点):

$$\hat{\boldsymbol{\theta}}_{\text{MLE}} = \arg \max_{\boldsymbol{\theta}} \sum_{i=1}^n \log P(\mathbf{x}_i | \boldsymbol{\theta})$$

详细推导示例——高斯分布的 MLE:

给定数据 $\{x_1, x_2, \dots, x_n\}$, 假设来自高斯分布 $N(\mu, \sigma^2)$

$$\mathcal{L}(\mu, \sigma^2) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_i - \mu)^2}{2\sigma^2}\right)$$

取对数:

$$\ell(\mu, \sigma^2) = -\frac{n}{2} \log(2\pi) - \frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2$$

对 μ 求偏导并令其为 0:

$$\frac{\partial \ell}{\partial \mu} = \frac{1}{n} \sum_{i=1}^n (x_i - \mu)$$

$$\Rightarrow \hat{\mu}_{MLE} = \frac{1}{n} \sum_{i=1}^n x_i$$

对 σ^2 求偏导并令其为0：

$$\frac{\partial \ell}{\partial \sigma^2} = -\frac{1}{2(\sigma^2)^2} \sum_{i=1}^n (x_i - \mu)^2$$

$$\Rightarrow \hat{\sigma}^2_{MLE} = \frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2$$

```
import numpy as np
```

MLE估计高斯分布参数

```
np.random.seed(42)
```

```
true_mu, true_sigma = 5.0, 2.0
```

```
data = np.random.normal(true_mu, true_sigma, 1000)
```

MLE估计

```
mumle = data.mean()
```

```
sigma2mle = ((data - mumle) ** 2).mean() # 注意：除以n，不是n-1
```

```
print(f"真实值: μ={true_mu}, σ²={true_sigma**2}")
```

```
print(f"MLE估计: μ̂={mumle:.3f}, σ̂²={sigma2mle:.3f}")
```

注意：MLE的方差估计是有偏的

无偏估计： $\hat{\sigma}^2_{unbiased} = \frac{\sum (x_i - \bar{x})^2}{(n-1)}$

```
sigma2unbiased = data.var(ddof=1)
```

```
print(f"无偏估计: σ̂²={sigma2unbiased:.3f}")
```

当n很大时，两者几乎相同

极大后验估计 (MAP)

目标：在似然的基础上加入先验，找到使后验概率最大的参数值。

$$\hat{\theta}_{MAP} = \arg \max_{\theta} P(\theta | \mathbf{X}) = \arg \max_{\theta} \left\{ \frac{P(\theta) P(\mathbf{X} | \theta)}{P(\mathbf{X})} \right\}$$

由于 $P(\mathbf{X})$ 与 θ 无关：

$$\hat{\theta}_{\text{MAP}} = \arg\max_{\theta} [P(\mathbf{X}|\theta)]$$

MAP与正则化的关系：

先验分布 等价的正则化 解释

高斯先验 $P(\theta) \sim N(0, \sigma^2)$ L2正则化（岭回归） 参

拉普拉斯先验 $P(\theta) \sim \text{Laplace}(0, b)$ L1正则化
参数趋向于0，且可以精确为0（稀疏性）

推导：MAP $\rightarrow$ L2正则化

设先验 $P(\theta) = \prod_j \frac{1}{\sqrt{2\pi}\sigma_0} \exp(-\frac{\theta_j^2}{2\sigma_0^2})$ ，则：

$$\log P(\theta|\mathbf{X}) \propto \log P(\mathbf{X}|\theta) - \frac{1}{2\sigma_0^2} \sum_j \theta_j^2$$

最大化等价于最小化：

$$-\log P(\mathbf{X}|\theta) + \frac{1}{2\sigma_0^2} \sum_j \theta_j^2$$

这正是 损失函数 + L2正则化项 的形式，其中正则化系数 $\lambda = \frac{1}{\sigma_0^2}$

```
import numpy as np
```

MAP估计 vs MLE估计

示例：抛硬币，观测到3次正面7次反面

MLE估计

```
nheads, ntails = 3, 7
thetamle = nheads / (nheads + ntails)
print(f"MLE估计:  $\theta^* = \{\text{thetamle:.3f}\}$ ") # 0.300
```

MAP估计（使用Beta(2, 2)先验——偏向0.5的先验）

```
alphaprior, betaprior = 2, 2
thetamap = (nheads + alphaprior - 1) / (nheads +
```

```
print(f"MAP估计:  $\theta^{\wedge}$  = {thetamap:.3f}") # 0.333
```

MAP估计 (使用 $\text{Beta}(5, 5)$ 先验——更强的偏向0.5的先验)

```
alphaprior, betaprior = 5, 5
```

```
thetamapstrong = (nheads + alphaprior - 1) / (nheads + betaprior - 1)
```

```
print(f"MAP估计(强先验):  $\theta^{\wedge}$  = {thetamapstrong:.3f}") # 0.5
```

直觉: 先验越强, 估计越偏向先验的期望 (0.5)

先验越弱 ($\alpha, \beta \rightarrow 1$), MAP越接近MLE

数据越多, 先验的影响越小 (数据 "淹没" 先验)

2.6 蒙特卡洛方法

核心思想: 通过随机采样来近似计算期望值。

$$E[f(X)] = \int f(x)p(x)dx \approx \frac{1}{N} \sum_{i=1}^N f(x^{(i)})$$

其中 $x^{(i)} \sim p(x)$ 。

大数定律保证: 当 $N \rightarrow \infty$ 时, 样本均值收敛到期望值。

应用1: 估计 π

```
import numpy as np
```

```
def estimatepi(nsamples=1000000):
```

```
    """用蒙特卡洛方法估计 $\pi$ """
```

```
    # 在单位正方形  $[0, 1] \times [0, 1]$  中随机撒点
```

```
    x = np.random.uniform(0, 1, nsamples)
```

```
    y = np.random.uniform(0, 1, nsamples)
```

```
    # 计算落在单位圆内的比例
```

```
    inside = (x2 + y2) <= 1
```

```
    #  $\pi/4 \approx \text{圆内面积} / \text{正方形面积}$ 
```

```
    piestimate = 4 * inside.mean()
```

```
    return piestimate
```

```
print(f"π的估计值: {estimate_pi():.4f}") # ≈ 3.1416
```

应用2：贝叶斯推断中的后验采样

```
import numpy as np
```

```
def montecarloposterior(priorsamples, likelihoodf):
    """
```

用蒙特卡洛方法估计后验期望

$$E[\theta \mid \text{data}] \approx (1/N) \sum \theta_i w_i / \sum w_j$$

其中 $w_i = P(\text{data} \mid \theta_i)$ 是似然权重

```
"""
```

```
# 从先验采样
```

```
thetasamples = priorsamples(nsamples)
```

```
# 计算每个样本的似然权重
```

```
weights = np.array([likelihoodfn(data, theta) for theta in thetasamples])
```

```
# 归一化权重
```

```
weights = weights / weights.sum()
```

```
# 加权平均就是后验期望的估计
```

```
posteriormean = np.average(thetasamples, weights=weights)
```

```
return posteriormean
```

2.7 马尔可夫链与MCMC

马尔可夫链：一种随机过程，未来状态只依赖当前状态，不依赖过去的历史（无记忆性）。

$$P(X_{t+1} \mid X_t, X_{t-1}, \dots, X_0) = P(X_{t+1} \mid X_t)$$

平稳分布：如果马尔可夫链运行足够长时间，其状态分布会收敛到一个固定分布 π ，满足：

$$\pi = \pi \cdot P$$

其中 P 是转移矩阵。

MCMC（马尔可夫链蒙特卡洛）：构造一个马尔可夫链，使其平稳分布就是我们想采样的目标分布，然后从该链中采样。

Metropolis-Hastings 算法：

```
import numpy as np
from scipy import stats

def metropolishastings(targetdensity, proposalstd):
    """
    Metropolis-Hastings MCMC 采样器

    targetdensity: 目标分布的密度函数（可以未归一化）
    proposalstd: 提议分布的标准差
    """
    samples = []
    current = 0 # 初始状态

    for i in range(nsamples + burnin):
        # 步骤1：从提议分布中生成候选样本
        # 这里使用对称的高斯分布作为提议分布
        proposed = current + np.random.normal(0, proposalstd)

        # 步骤2：计算接受概率
        #  $\alpha = \min(1, p(\text{proposed}) / p(\text{current}))$ 
        pproposed = targetdensity(proposed)
        pcurrent = targetdensity(current)

        if pcurrent == 0:
            acceptanceprob = 1
        else:
            acceptanceprob = min(1, pproposed / pcurrent)

        # 步骤3：决定是否接受
        if np.random.random() < acceptanceprob:
            current = proposed # 接受：移动到新状态
```

否则停留在当前状态

```
if i >= burnin:
    samples.append(current)

return np.array(samples)
```

示例：从一个双峰分布中采样

```
def bimodal_density(x):
    """ 双峰分布：两个高斯分布的混合 """
    return 0.3 * stats.norm.pdf(x, -3, 1) + 0.7 * stats.norm.pdf(x, 3, 1)

samples = metropolis_hastings(bimodal_density, prop, 10000)
print(f"采样均值: {samples.mean():.2f}") # 应该在两个峰之间的某处
print(f"采样标准差: {samples.std():.2f}")
```

MCMC在AI中的应用：

- 贝叶斯神经网络：从参数后验分布中采样
- 概率编程：Stan, PyMC等框架的核心
- LDA主题模型：Gibbs采样推断主题
- 扩散模型：反向采样过程可以看作MCMC

2.8 信息论基础

概念 公式 AI应用

信息熵 $H(X) = -\sum p(x) \log p(x)$ 决策树分裂、不确定性度量

交叉熵 $H(p, q) = -\sum p(x) \log q(x)$ 分类损失函数

KL散度 $D_{KL}(p \parallel q) = \sum p(x) \log \frac{p(x)}{q(x)}$

互信息 $I(X; Y) = H(X) - H(X \setminus Y)$ 特征选择

交叉熵作为损失函数的详细解释：

在分类任务中，真实分布 p 是 one-hot 向量，模型预测分布为 q ：

$$H(p, q) = -\sum_i p_i \log q_i = -\log q\{y\{\text{true}\}\}$$

这就是负对数似然（NLL）损失。最小化交叉熵等价于最大化似然。

```
import torch
import torch.nn as nn
```

交叉熵损失的直觉

假设真实标签是类别 2，模型预测概率分布为 $[0.1, 0.2, 0.6, 0.1]$

好的预测（概率集中在正确类别）→ 交叉熵小

```
goodpred = torch.tensor([0.1, 0.2, 0.6, 0.1])
lossgood = -torch.log(goodpred[2]) # -log(0.6) ≈
```

差的预测（概率分散）→ 交叉熵大

```
badpred = torch.tensor([0.25, 0.25, 0.25, 0.25])
lossbad = -torch.log(badpred[2]) # -log(0.25) ≈ 1
```

很差的预测（概率在错误类别）→ 交叉熵很大

```
worstpred = torch.tensor([0.6, 0.2, 0.1, 0.1])
lossworst = -torch.log(worstpred[2]) # -log(0.1)
```

```
print(f"好预测的损失: {lossgood:.2f}")
print(f"差预测的损失: {lossbad:.2f}")
print(f"很差预测的损失: {lossworst:.2f}")
```

- - -

三、最优化方法：AI 模型的训练基础

最优化方法是 AI 技术的核心驱动力。无论是训练神经网络，还是调优模型参数，最优化方法都扮演着至关重要的角色。

3.1 凸优化基础

凸集：集合 C 是凸集，当且仅当对于任意 $x, y \in C$ 和 $\lambda \in [0, 1]$

$$\lambda x + (1 - \lambda)y \in C$$

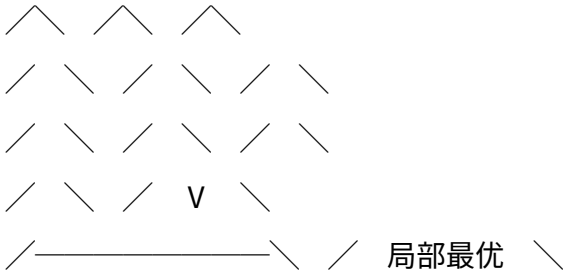
直觉：集合中任意两点的连线段都在集合内。

凸函数：函数 f 是凸函数，当且仅当其定义域是凸集，且对于任意 x, y 和 $\lambda \in [0, 1]$

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y)$$

直觉：函数值在任意两点之间的连线段位于函数图像下方。

凸函数： 非凸函数：



全局最优 但不是全局最优

凸优化的关键性质：

1. 局部最优 = 全局最优（最重要的性质！）
2. 如果 f 是严格凸的，则全局最优点唯一
3. KKT条件是充要条件（非凸问题中KKT只是必要条件）

```
import numpy as np
```

判断凸性的方法

方法1：二阶条件

对于二次可微函数 f ， f 是凸函数 $\iff$ Hessian矩阵 H 处处半正定
即： $\forall x, H(x) \succeq 0$ （所有特征值 ≥ 0 ）

```
def isconvexbyhessian(f, x, eps=1e-5):
    """通过检查Hessian矩阵是否半正定来判断凸性"""
    n = len(x)
    H = np.zeros((n, n))

    # 数值计算Hessian
    fx = f(x)
    for i in range(n):
        for j in range(i, n):
            # 二阶偏导数的中心差分
            xpp = x.copy(); xpp[i] += eps; xpp[j] += eps
```

```

xpm = x.copy(); xpm[i] += eps; xpm[j] -= eps
xmp = x.copy(); xmp[i] -= eps; xmp[j] += eps
xmm = x.copy(); xmm[i] -= eps; xmm[j] -= eps

H[i, j] = (f(xpp) - f(xpm) - f(xmp) + f(xmm)) / (eps**2)
H[j, i] = H[i, j]

# 检查是否半正定：所有特征值 ≥ 0
eigenvalues = np.linalg.eigvalsh(H)
return np.all(eigenvalues >= -1e-10), eigenvalues

```

示例

```

fconvex = lambda x: x[0]**2 + x[1]**2 # 凸函数
fnonconvex = lambda x: x[0]**4 - x[0]**2 + x[1]**2 # 非凸函数

iscvx, eigs = isconvexbyhessian(fconvex, np.array([1, 1]))
print(f"x^2 + y^2 在(1, 1)处: 凸={iscvx}, 特征值={eigs}")

iscvx, eigs = isconvexbyhessian(fnonconvex, np.array([0, 0]))
print(f"x^4 - x^2 + y^2 在(0, 0)处: 凸={iscvx}, 特征值={eigs}")

```

3.2 梯度下降：详细数学推导

梯度下降更新规则：

$$\theta^{t+1} = \theta^t - \alpha \nabla f(\theta^t)$$

为什么沿负梯度方向？——最速下降法推导

问题：在当前点 θ^t ，应该沿什么方向 d 移动，使函数值下降最快？

约束优化：

$$\min_d \nabla f(\theta^t)^T d \quad \text{s.t.} \quad \|d\| = 1$$

由Cauchy-Schwarz不等式：

$$\nabla f(\theta^t)^T d \geq -\|\nabla f(\theta^t)\|$$

等号成立当且仅当 $d = -\nabla f(\theta) / \|\nabla f(\theta)\|$

所以负梯度方向是函数值下降最快的方向。

收敛性分析——凸函数的情况：

定理*：如果 f 是凸函数， L - Lipschitz连续梯度 ($\|\nabla f(x)\| \leq L$)，则梯度下降以步长 $\alpha = 1/L$ 收敛：

$$f(\theta_t) - f(\theta^*) \leq \frac{L}{2t} \|\theta_0 - \theta^*\|^2$$

收敛速度为 $O(1/t)$ ——次线性收敛。

强凸函数的情况 ($f = \frac{\mu}{2} \|\cdot\|^2$ 仍为凸)：

$$f(\theta_t) - f(\theta^*) \leq \left(1 - \frac{\mu}{L}\right)^t \left(f(\theta_0) - f(\theta^*)\right)$$

收敛速度为线性收敛——指数级衰减！

```
import numpy as np
```

梯度下降的完整实现和可视化

```
class GradientDescent:
    def __init__(self, lr=0.01, max_iter=1000, tol=1e-6):
        self.lr = lr
        self.max_iter = max_iter
        self.tol = tol

    def minimize(self, f, gradf, x0):
        """ 最小化函数 f，使用其梯度 gradf """
        x = x0.copy()
        history = [x.copy()]

        for i in range(self.max_iter):
            g = gradf(x)

            # 更新
            x_new = x - self.lr * g
```

```

# 收敛判断
if np.linalg.norm(xnew - x) < self.tol:
    break

x = xnew
history.append(x.copy())

return x, np.array(history)

```

示例：Rosenbrock函数（经典的优化测试函数）

```

def rosenbrock(x):
    return (1 - x[0])**2 + 100 * (x[1] - x[0]**2)**2

def gradrosenbrock(x):
    dx = -2 * (1 - x[0]) - 400 * x[0] * (x[1] - x[0]**2)
    dy = 200 * (x[1] - x[0]**2)
    return np.array([dx, dy])

gd = GradientDescent(lr=0.001, maxiter=10000)
xopt, history = gd.minimize(rosenbrock, gradrosenbrock)
print(f"最优解: {xopt}") # 应接近 [1, 1]
print(f"最优值: {rosenbrock(xopt):.6f}") # 应接近 0

```

3.3 牛顿法和拟牛顿法

牛顿法

核心思想：在当前点用二次函数近似目标函数，然后直接跳到二次函数的最小值。

推导：对 $f(\theta)$ 在 θ^t 处进行二阶Taylor展开：

$$f(\theta) \approx f(\theta^t) + \nabla f(\theta^t)^T (\theta - \theta^t) + \frac{1}{2} (\theta - \theta^t)^T H(\theta^t) (\theta - \theta^t)$$

对近似函数求导并令其为0：

$$\nabla f(\theta^t) + H(\theta^t) (\theta - \theta^t) = 0$$

$$\theta_{t+1} = \theta_t - H(\theta_t)^{-1} \nabla f(\theta_t)$$

牛顿法的优缺点：

方面 梯度下降 牛顿法

收敛速度 线性（凸）或次线性 二次收敛（在最优解附近）

每步计算量 $O(n)$ 计算梯度 $O(n^2)$ 计算Hessian, $O(n^3)$ 求逆

步长 需要调整学习率 自然的步长（Hessian的逆决定）

内存 $O(n)$ $O(n^2)$ 存储Hessian

适用规模 任意规模 中小规模 ($n < 10^4$)

非凸问题 * 可以使用 Hessian可能不正定，需要修正

```
import numpy as np

def newtonmethod(f, gradf, hessianf, x0, maxiter=1000):
    """ 牛顿法优化 """
    x = x0.copy().astype(float)

    for i in range(maxiter):
        g = gradf(x)
        H = hessianf(x)

        # 检查Hessian是否正定
        eigenvalues = np.linalg.eigvalsh(H)
        if np.any(eigenvalues <= 0):
            # 如果不正定，添加阻尼项使其正定
            H += (-eigenvalues.min() + 1e-4) * np.eye(len(x))

        # 牛顿步： $\Delta x = -H^{-1} g$ 
        deltax = np.linalg.solve(H, -g)

        # 线搜索（确保函数值下降）
        step = 1.0
        while f(x + step * deltax) > f(x):
            step = 0.5
```

```

if step < 1e-10:
    break

xnew = x + step * delta x

if np.linalg.norm(xnew - x) < tol:
    break

x = xnew

return x

```

示例

```

f = lambda x: x[0]**2 + 3*x[1]**2
gradf = lambda x: np.array([2*x[0], 6*x[1]])
hessianf = lambda x: np.array([[2, 0], [0, 6]])

xopt = newtonmethod(f, gradf, hessianf, np.array([0, 0]))
print(f"牛顿法最优解: {xopt}") # 应该是 [0, 0]

```

L - BFGS (拟牛顿法)

L - BFGS (Limited-memory BFGS) 是牛顿法的实用近似:

- 不需要存储完整的 $n \times n$ Hessian 矩阵
- 只存储最近 m 次迭代的梯度差和位移差 (m 通常为 5 - 20)
- 内存从 $O(n^2)$ 降到 $O(mn)$
- 适合中等规模的优化问题

```

from scipy.optimize import minimize
import numpy as np

```

使用 L - BFGS - B 优化 (带边界约束的 L - BFGS)

```

def rosenbrock(x):
    return (1 - x[0])**2 + 100 * (x[1] - x[0]**2)**2

def gradrosenbrock(x):
    dx = -2 * (1 - x[0]) - 400 * x[0] * (x[1] - x[0]**2)

```


$$\sum_{i=1}^n \alpha_i [y_i (\mathbf{w}^T \mathbf{x}_i +$$

对 $\mathbf{w}$ 和 b 求偏导并令其为 0：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_i} = 0 \Rightarrow \mathbf{w} = \sum_{i=1}^n \alpha_i \mathbf{x}_i$$

$$\frac{\partial \mathcal{L}}{\partial b} = -\sum_{i=1}^n \alpha_i y_i$$

代回得到对偶问题：

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j \mathbf{x}_i^T \mathbf{x}_j$$

$$\text{s.t.} \quad \alpha_i \geq 0, \quad \sum_{i=1}^n \alpha_i y_i = 0$$

互补松弛性告诉我们：只有支持向量（恰好在间隔边界上的点）对应的 $\alpha_i > 0$ ，其他点 $\alpha_i = 0$ 。这就是为什么 SVM 是稀疏的——决策只依赖少数支持向量。

```
import numpy as np
from cvxopt import matrix, solvers
```

```
def svmtrain(X, y):
    """
```

SVM 的完整训练过程（使用对偶问题求解）

X : (nsamples, nfeatures) 训练数据

y : (nsamples,) 标签 (+1 或 -1)

```
"""
```

```
n = len(y)
```

```
# 构建对偶问题的矩阵
```

```
# min 0.5 \alpha^T Q \alpha - e^T \alpha
```

```
# s.t. y^T \alpha = 0, 0 \le \alpha_i \le C
```

```
K = X @ X.T # 核矩阵（线性核）
```

```
Q = matrix(np.outer(y, y) * K)
```

```
e = matrix(-np.ones(n))
```

```
# 等式约束：y^T \alpha = 0
```

```

A = matrix(y.astype(float).reshape(1, -1))
b = matrix(0.0)

# 不等式约束： $\alpha_i \geq 0$  (通过对角矩阵G和向量h实现)
G = matrix(np.vstack([-np.eye(n), np.eye(n)]))
h = matrix(np.hstack([np.zeros(n), np.ones(n)]))

# 求解
solution = solvers.qp(Q, e, G, h, A, b)
alpha = np.array(solution['x']).flatten()

# 找到支持向量
svmask = alpha > 1e-5

# 计算权重
w = np.sum(alpha[svmask, None] * y[svmask, None] * X[svmask, :])

# 计算偏置
b = y[svmask][0] - w @ X[svmask][0]

return w, b, svmask

```

3.5 Adam优化器：完整算法与直觉解释

Adam (Adaptive Moment Estimation, Kingma & Ba, 2015) 是目前最常用的优化器，它结合了Momentum和RMSProp的优点。

完整算法：

输入：学习率 α ，一阶矩衰减率 $\beta_1 = 0.9$ ，二阶矩衰减率 $\beta_2 = 0.999$ ， $\epsilon = 10^{-8}$
 初始化： $m_0 = 0$ ， $v_0 = 0$ ， $t = 0$

重复：

```

t ← t + 1
g ← ∇f(θ{t-1}) # 计算梯度
m_t ← β1 · m{t-1} + (1 - β1) · g # 更新一阶矩（梯度的指数移动平均）
v_t ← β2 · v{t-1} + (1 - β2) · g2 # 更新二阶矩（梯度平方的指数移动平均）
m̂_t ← m_t / (1 - β1t) # 偏差修正（因为m0 = 0，初期估计偏低）

```

$$\hat{v}_t \leftarrow v_t / (1 - \beta_2^t) \quad \# \text{ 偏差修正}$$

$$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) \quad \# \text{ 参数更新}$$

直到收敛

为什么需要偏差修正？

初始化 $m_0 = 0$ ，所以初期的移动平均偏向0。例如，第一步的移动平均为：

$$m_1 = \beta_1 \cdot 0 + (1 - \beta_1) \cdot g_1 = (1 - \beta_1) g_1$$

而不是我们期望的 g_1 。偏差修正确保初期估计无偏：

$$\hat{m}_1 = \frac{m_1}{1 - \beta_1^1} = \frac{0.1}{1 - 0.9} = 1.0$$

```
import numpy as np
```

```
class Adam:
```

```
    """Adam优化器的完整实现"""
```

```
    def __init__(self, params, lr=0.001, betas=(0.9, 0.999),
```

```
               self.lr = lr
```

```
               self.beta1, self.beta2 = betas
```

```
               self.eps = eps
```

```
               self.t = 0
```

```
        # 为每个参数初始化一阶矩和二阶矩
```

```
        self.m = np.zeros_like(params)
```

```
        self.v = np.zeros_like(params)
```

```
    def step(self, params, grads):
```

```
        """执行一步参数更新"""
```

```
        self.t += 1
```

```
        # 更新一阶矩（梯度的指数移动平均 —— "动量"）
```

```
        self.m = self.beta1 * self.m + (1 - self.beta1) * grads
```

```
        # 更新二阶矩（梯度平方的指数移动平均 —— "自适应学习率"）
```

```
        self.v = self.beta2 * self.v + (1 - self.beta2) * (grads * grads)
```



```

# 偏差修正
m_hat = self.m / (1 - self.beta1*self.t)
v_hat = self.v / (1 - self.beta2*self.t)

# 参数更新
params -= self.lr * m_hat / (np.sqrt(v_hat) + self.epsilon)

return params

```

使用示例

```

np.random.seed(42)
params = np.random.randn(100) # 模拟参数
optimizer = Adam(params, lr=0.001)

```

模拟训练循环

```

for step in range(1000):
    grads = 2 * params + np.random.randn(100) * 0.1 # 模拟梯度
    params = optimizer.step(params, grads)

    if step % 200 == 0:
        loss = np.sum(params**2)
        print(f"Step {step}, Loss: {loss:.4f}")

```

Adam的直观理解：

- 一阶矩

m （动量）：累积了过去的梯度方向。好处是在“斜坡”上加速，在“谷底”附近减速。类比：一个球在坡上滚动。

- 二阶矩

v （自适应学习率）：累积了过去的梯度大小。好处是：梯度大的参数用更小的学习率（更谨慎），梯度小的参数用更大的学习率。

- 偏差修正：让初期（前几百步）的估计更准确。随着 $t \rightarrow \infty$ ，修正因子 $\frac{1}{t}$ 趋近于0，修正的影响消失。

3.6 反向传播与梯度计算

反向传播的基本步骤：

1. 前向传播：计算输出和损失

2 . 反向传播：计算梯度

3 . 权重更新*：根据梯度调整权重

```
import numpy as np
```

定义激活函数

```
def sigmoid(x):  
    return 1 / (1 + np.exp(-x))
```

```
def sigmoidderiv(x):  
    s = sigmoid(x)  
    return s * (1 - s)
```

定义损失函数

```
def mse_loss(ypred, ytrue):  
    return np.mean((ypred - ytrue) ** 2)
```

初始化权重和偏置

```
np.random.seed(0)  
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])  
y = np.array([[0], [1], [1], [0]])
```

使用更深的网络解决XOR问题

```
W1 = np.random.randn(2, 4) * 0.5  
b1 = np.zeros((1, 4))  
W2 = np.random.randn(4, 1) * 0.5  
b2 = np.zeros((1, 1))
```

```
learningrate = 0.5  
epochs = 5000
```

```
for epoch in range(epochs):
```

```
# 前向传播
```

```
z1 = X @ W1 + b1  
a1 = sigmoid(z1)  
z2 = a1 @ W2 + b2
```

```

y_pred = sigmoid(z2)

# 计算损失
loss = mseloss(y_pred, y)

# 反向传播
# 输出层梯度
dz2 = (y_pred - y) * sigmoidderiv(z2)
dW2 = a1.T @ dz2 / 4
db2 = dz2.mean(axis=0, keepdims=True)

# 隐藏层梯度
da1 = dz2 @ W2.T
dz1 = da1 * sigmoidderiv(z1)
dW1 = X.T @ dz1 / 4
db1 = dz1.mean(axis=0, keepdims=True)

# 更新权重
W2 -= learningrate * dW2
b2 -= learningrate * db2
W1 -= learningrate * dW1
b1 -= learningrate * db1

if epoch % 1000 == 0:
    print(f"Epoch {epoch}, Loss: {loss:.4f}")

测试
print("\n预测结果:")
print(y_pred.round(3))

```

3.7 常见的最优化算法

算法 特点 适用场景

SGD 随机梯度下降，简单高效 大规模数据，初期探索

Momentum 引入动量加速收敛 深层网络，损失面有"沟壑"

RMSProp 自适应学习率 RNN训练，非平稳目标

Adam 结合动量 + 自适应学习率 通用，最常用

AdamW Adam + 权重衰减解耦 Transformer 训练，更好的泛化

Nadam Adam + Nesterov 动量 需要更快收敛

AdamW vs Adam 的区别（重要！）：

Adam 的权重衰减实现（L2 正则化方式）：

$$\text{loss} = \text{originalloss} + \lambda \sum \theta^2$$
$$\text{gradient} = \nabla \text{originalloss} + 2\lambda \theta$$

问题：梯度被自适应学习率缩放，导致权重衰减的效果不一致

AdamW 的权重衰减实现（解耦方式）：

$$\theta_t = \theta_{t-1} - \alpha \cdot \hat{m} / (\sqrt{\hat{v}} + \epsilon) - \alpha \cdot \lambda \cdot \theta_{t-1}$$

↑ Adam 更新 ↑ 独立的权重衰减

好处：权重衰减不受自适应学习率的影响，更加稳定

PyTorch 中的使用

```
import torch
```

旧方式（不推荐）：Adam + weightdecay → 实际是 L2 正则化

```
optimizeradam = torch.optim.Adam(model.parameters)
```

新方式（推荐）：AdamW → 真正的解耦权重衰减

```
optimizeradamw = torch.optim.AdamW(model.parameters)
```

...

四、微积分：连续变化的数学

4.1 偏导数和梯度的几何直觉

偏导数：函数沿某个坐标轴方向的变化率。

$$\frac{\partial f}{\partial x_i} = \lim_{h \rightarrow 0} \frac{f(x_1, \dots, x_i + h, \dots, x_n) - f(x_1, \dots, x_i, \dots, x_n)}{h}$$

梯度：所有偏导数组成的向量。

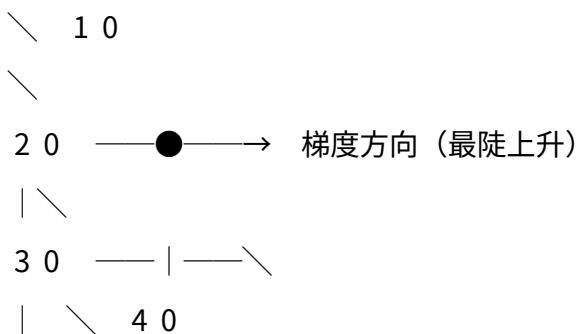
$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right)$$

$$\frac{\partial f}{\partial x_n} \text{ right)} \quad \quad$$

几何直觉：

- 梯度指向函数值上升最快的方向
- 负梯度指向函数值下降最快的方向
- 梯度与等高线（等值面）垂直
- 梯度的大小表示上升的速率

等高线图上的梯度方向：



梯度垂直于等高线，指向 " 上坡 " 方向

4 . 2 链式法则与反向传播

链式法则：复合函数的导数等于各层导数的乘积。

如果 $y = f(g(x))$ ，则：

$$\frac{dy}{dx} = \frac{dy}{dg} \cdot \frac{dg}{dx}$$

多元链式法则：

如果 $z = f(x, y)$ ，且 $x = g(t)$ ， $y = h(t)$ ，则：

$$\frac{dz}{dt} = \frac{\partial z}{\partial x} \frac{dx}{dt} + \frac{\partial z}{\partial y} \frac{dy}{dt}$$

链式法则与反向传播的关系：

反向传播本质上就是链式法则在计算图上的高效应用。

计算图示例： $z = f(g(h(x)))$

前向传播： $x \rightarrow h(x) \rightarrow g(h) \rightarrow f(g)$

反向传播： $\partial f / \partial g \rightarrow \partial g / \partial h \rightarrow \partial h / \partial x$ (链式法则逐层反向)

```
import numpy as np
```

示例：一个简单的两层神经网络

```
y = σ(W2 · σ(W1 · x + b1) + b2)
```

```
def forwardandbackward(x, ytrue, W1, b1, W2, b2):
```

```
    """完整的前向和反向传播"""
```

```
# 前向传播（记录中间值）
```

```
z1 = x @ W1 + b1 # 线性变换1
```

```
a1 = 1 / (1 + np.exp(-z1)) # 激活1 (sigmoid)
```

```
z2 = a1 @ W2 + b2 # 线性变换2
```

```
a2 = 1 / (1 + np.exp(-z2)) # 激活2 (sigmoid)
```

```
loss = np.mean((a2 - ytrue) * 2) # MSE损失
```

```
# 反向传播（链式法则）
```

```
# Loss = MSE(a2, y) = mean((a2 - y)^2)
```

```
# 第1步：损失对输出的梯度
```

```
da2 = 2 * (a2 - ytrue) / len(ytrue) # ∂Loss / ∂a2
```

```
# 第2步：通过sigmoid的梯度（链式法则）
```

```
dz2 = da2 * a2 * (1 - a2) # ∂Loss / ∂z2 = ∂Loss / ∂a2 · da2
```

```
# 第3步：通过线性变换的梯度
```

```
dW2 = a1.T @ dz2 # ∂Loss / ∂W2 = ∂Loss / ∂z2 · ∂z2 / ∂W2
```

```
db2 = dz2.sum(axis=0) # ∂Loss / ∂b2
```

```
da1 = dz2 @ W2.T # ∂Loss / ∂a1 (传给下一层)
```

```
# 第4步：通过sigmoid的梯度
```

```
dz1 = da1 * a1 * (1 - a1) # ∂Loss / ∂z1
```

```
# 第5步：通过线性变换的梯度
```

```

dW1 = x.T @ dz1 # ∂Loss / ∂W1
db1 = dz1.sum(axis=0) # ∂Loss / ∂b1

return loss, {'W1': dW1, 'b1': db1, 'W2': dW2, 'b2': db2}

```

链式法则的直觉：

每一层的梯度 = 上游梯度 × 本层局部梯度

这就是 " 反向传播 " ——梯度从输出层向输入层反向传播

4.3 Jacobian 矩阵

对于向量值函数 $\mathbf{f}: \mathbb{R}^n \rightarrow \mathbb{R}^m$ ，Jacobian 矩阵是所有一阶偏导数的矩阵：

$$J_{\mathbf{f}} = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \dots & \frac{\partial f_m}{\partial x_n} \end{pmatrix}$$

Jacobian 在深度学习中的应用：

1. 梯度检查：用数值 Jacobian 验证反向传播的实现
2. 可逆残差网络 (RevNet)：利用 Jacobian 行列式 = 1 的性质实现内存高效训练
3. 标准化流 (Normalizing Flows)：用 Jacobian 行列式计算概率密度变换

4.4 Hessian 矩阵

对于标量函数 $f: \mathbb{R}^n \rightarrow \mathbb{R}$ ，Hessian 矩阵

$$H_f = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \dots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \dots & \frac{\partial^2 f}{\partial x_n^2} \end{pmatrix}$$

Hessian 的 AI 应用：

应用 说明

最优学习率 对二次函数，最优步长 $\alpha = 1 / \lambda_{\max}$ (最大特征值的倒数)

鞍点检测 Hessian 正定 = 局部最小，负定 = 局部最大，不定 = 鞍点

自然梯度 用 Fisher 信息矩阵 (Hessian 的期望) 调整梯度方向

二阶优化 Newton 法使用 $H^{-1} \nabla f$ 作为搜索方向

Laplace近似 用Hessian的逆近似后验分布的协方差

```
import numpy as np

def compute_hessian(f, x, eps=1e-5):
    """数值计算Hessian矩阵"""
    n = len(x)
    H = np.zeros((n, n))
    f0 = f(x)

    for i in range(n):
        for j in range(i, n):
            # 使用中心差分计算二阶偏导
            xpp = x.copy(); xpp[i] += eps; xpp[j] += eps
            xpm = x.copy(); xpm[i] += eps; xpm[j] -= eps
            xmp = x.copy(); xmp[i] -= eps; xmp[j] += eps
            xmm = x.copy(); xmm[i] -= eps; xmm[j] -= eps

            H[i, j] = (f(xpp) - f(xpm) - f(xmp) + f(xmm)) / (4 * eps ** 2)
            H[j, i] = H[i, j]

    return H
```

判断临界点的性质

```
def classify_critical_point(H):
    eigenvalues = np.linalg.eigvalsh(H)
    if np.all(eigenvalues > 0):
        return "局部最小值"
    elif np.all(eigenvalues < 0):
        return "局部最大值"
    else:
        return "鞍点"
```

深度学习中的鞍点问题

高维空间中，几乎所有的临界点都是鞍点，而非局部最小值

这是因为：n维空间中，所有特征值同号的概率随n指数级下降

所以：梯度下降的主要障碍不是局部最小值，而是鞍点和平坦区域

- - -

五、统计学基础

5.1 假设检验

核心思想：在统计证据的基础上，判断某个假设是否成立。

假设检验的步骤：

1. 建立假设：

- 零假设 H_0 ：默认假设（如 " 模型 A 和模型 B 性能相同 "）
- 备择假设 H_1 ：我们想证明的假设（如 " 模型 B 优于模型 A "）

2. 选择检验统计量：如 t 统计量、 χ^2 统计量等

3. 计算 p 值：在 H_0 成立的条件下，观测到当前或更极端结果的概率

4. 做决策：如果 $p < \alpha$ （通常 $\alpha = 0.05$ ），则拒绝 H_0

```
import numpy as np
from scipy import stats

def twomodelcomparison(accuracyA, accuracyB, ntests):
    """
```

比较两个模型准确率是否有显著差异（McNemar 检验的简化版）

更严格的做法是使用交叉验证 + 配对 t 检验

```
"""
```

```
# 假设我们做了多次实验
```

```
# 这里用多次独立测试的准确率来比较
```

```
# 配对 t 检验
```

```
tstat, pvalue = stats.ttestrel(accuracyA, accuracyB)
```

```
print(f"模型A均值: {np.mean(accuracyA):.4f}")
```

```

print ( f " 模型B均值:  { np.mean ( accuracyB ) : . 4 f } " )
print ( f " t统计量:  { tstat : . 4 f } " )
print ( f " p值:  { pvalue : . 4 f } " )

if pvalue < 0.05:
print ( " 结论: 两个模型有显著差异 (p < 0.05) " )
else:
print ( " 结论: 无法拒绝零假设——两个模型可能没有显著差异 " )

return pvalue

```

模拟实验

```

np.random.seed ( 42 )
accA = np.random.normal ( 0.85, 0.02, 30 ) # 模型A: 30次
accB = np.random.normal ( 0.87, 0.02, 30 ) # 模型B: 30次

twomodelcomparison ( accA, accB, ntestsamples = 1000 )

```

5.2 置信区间

定义: 参数的真实值以一定概率落在某个区间内。

$$P (\hat{\theta} - z_{\alpha/2} \cdot SE \leq \theta \leq \hat{\theta} + z_{\alpha/2} \cdot SE) = 1 - \alpha$$

其中 SE 是标准误差, $z_{\alpha/2}$ 是标准正态分布的分位数。

```

import numpy as np
from scipy import stats

def computeconfidenceinterval ( data, confidence = 0.95 ):
    """ 计算均值的置信区间 """
    n = len ( data )
    mean = np.mean ( data )
    se = stats.sem ( data ) # 标准误差 = s / sqrt(n)

    # t分布的分位数 (小样本用t分布而非正态分布)
    tcritical = stats.t.ppf ( ( 1 + confidence ) / 2, df = n - 1 )

```

```
margin = tcritical se
return mean - margin, mean + margin
```

示例：模型准确率的置信区间

```
np.random.seed(42)
accuracies = np.random.normal(0.85, 0.03, 50)

ci_low, ci_high = computeconfidenceinterval(accuracies)
print(f"准确率: {np.mean(accuracies):.3f}")
print(f"95%置信区间: [{ci_low:.3f}, {ci_high:.3f}]")
print(f"解释: 我们有95%的信心认为真实准确率在 [{ci_low:.3f}, {ci_high:.3f}]")
```

5.3 A / B测试

A / B测试是互联网产品决策的核心工具，本质上就是随机化对照实验 + 假设检验。

```
import numpy as np
from scipy import stats

def abtest(controlconversions, controltotal,
            treatmentconversions, treatmenttotal,
            alpha=0.05):
    """
```

A / B测试：比较两个版本的转化率

参数：

- controlconversions: 对照组转化数
 - controltotal: 对照组总人数
 - treatmentconversions: 实验组转化数
 - treatmenttotal: 实验组总人数
 - alpha: 显著性水平
- """

计算转化率

```
pcontrol = controlconversions / controltotal
ptreatment = treatmentconversions / treatmenttotal
```

```

# 合并转化率（零假设下的估计）
ppooled = (controlconversions + treatmentconversions) / (controltotal + treatmenttotal)

# 标准误差
se = np.sqrt(ppooled * (1 - ppooled) / (1/controltotal + 1/treatmenttotal))

# Z统计量
z = (ptreatment - pcontrol) / se

# p值（双侧检验）
pvalue = 2 * (1 - stats.norm.cdf(abs(z)))

# 置信区间
sediff = np.sqrt(pcontrol*(1-pcontrol)/controltotal + ptreatment*(1-ptreatment)/treatmenttotal)
cilow = (ptreatment - pcontrol) - 1.96 * sediff
cihigh = (ptreatment - pcontrol) + 1.96 * sediff

# 效应量（相对提升）
relativelift = (ptreatment - pcontrol) / pcontrol

print(f"对照组转化率: {pcontrol:.4f}")
print(f"实验组转化率: {ptreatment:.4f}")
print(f"绝对差异: {ptreatment - pcontrol:.4f}")
print(f"相对提升: {relativelift:.2%}")
print(f"95%置信区间: [{cilow:.4f}, {cihigh:.4f}]")
print(f"Z统计量: {z:.4f}")
print(f"p值: {pvalue:.4f}")

if pvalue < alpha:
    print(f"结论: 实验组显著优于对照组 (p = {pvalue:.4f} < {alpha})")
else:
    print(f"结论: 无法确认实验组优于对照组 (p = {pvalue:.4f} ≥ {alpha})")

```

```
print(" 可能需要更多样本量 ")
```

使用示例

```
abtest(
    controlconversions=150, controltotal=5000, # 对照组:
    treatmentconversions=185, treatmenttotal=5000, #
)
```

5.4 偏差 - 方差权衡 (Bias - Variance Tradeoff)

核心公式：

$$\text{MSE} = \text{Bias}^2 + \text{Variance} + \text{Noise}$$

其中：

- 偏差 (Bias)：模型预测值的期望与真实值之间的差距——衡量模型的准确性
- 方差 (Variance)：模型预测值随训练数据变化的波动程度——衡量模型的稳定性
- 噪声 (Noise)：数据本身的不可约误差——无法消除

数学推导：

$$E[(y - \hat{f}(x))^2] = E[(y - f(x) + f(x) - \hat{f}(x))^2]$$
$$= E[\epsilon^2] + E[(f(x) - \hat{f}(x))^2] + 2E[\epsilon(f(x) - \hat{f}(x))]$$

因为 ϵ 与 $\hat{f}$ 独立，第三项为0：

$$= \sigma^2 + E[(f(x) - E[\hat{f}(x)])^2] + E[(E[\hat{f}(x)] - \hat{f}(x))^2]$$
$$= \underbrace{\sigma^2}_{\text{噪声}} + \underbrace{E[(f(x) - E[\hat{f}(x)])^2]}_{\text{方差}}$$

```
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline
```

```

def biasvariancedemo():
    """
    演示偏差 - 方差权衡
    使用多项式回归拟合  $y = \sin(x) + \text{noise}$ 
    """
    np.random.seed(42)

    # 真实函数
    truefunc = lambda x: np.sin(2 * np.pi * x)

    # 不同复杂度的模型（多项式阶数）
    degrees = [1, 3, 9] # 欠拟合 → 合适 → 过拟合
    nexperiments = 100 # 重复实验次数
    ntrain = 20 # 训练样本数

    results = {}

    for degree in degrees:
        predictions = []

        for i in range(nexperiments):
            # 生成训练数据
            xtrain = np.sort(np.random.uniform(0, 1, ntrain))
            ytrain = truefunc(xtrain) + np.random.normal(0, 0.1, ntrain)

            # 训练模型
            model = Pipeline([
                ('poly', PolynomialFeatures(degree=degree)),
                ('linear', LinearRegression())
            ])
            model.fit(xtrain.reshape(-1, 1), ytrain)

            # 在测试点上预测
            xtest = np.linspace(0, 1, 100)
            ypred = model.predict(xtest.reshape(-1, 1))

```

```

predictions.append(ypred)

predictions = np.array(predictions)

# 计算偏差和方差
xtest = np.linspace(0, 1, 100)
ytrue = truefunc(xtest)

meanpred = predictions.mean(axis=0)
biassq = np.mean((meanpred - ytrue) ** 2)
variance = np.mean(predictions.var(axis=0))
totalerror = biassq + variance

results[degree] = {
    'biassq': biassq,
    'variance': variance,
    'total': totalerror
}

print(f"多项式阶数={degree}: Bias2={biassq:.4f}, Variance={variance:.4f}, Total={totalerror:.4f}")

return results

results = biasvariancedemo()

```

典型输出：

```

多项式阶数=1: Bias2=0.3000, Variance=0.0020, Total=0.3020
多项式阶数=3: Bias2=0.0050, Variance=0.0080, Total=0.0130
多项式阶数=9: Bias2=0.0001, Variance=0.0500, Total=0.0501

```

偏差 - 方差权衡的实践指导：

现象 诊断 解决方法

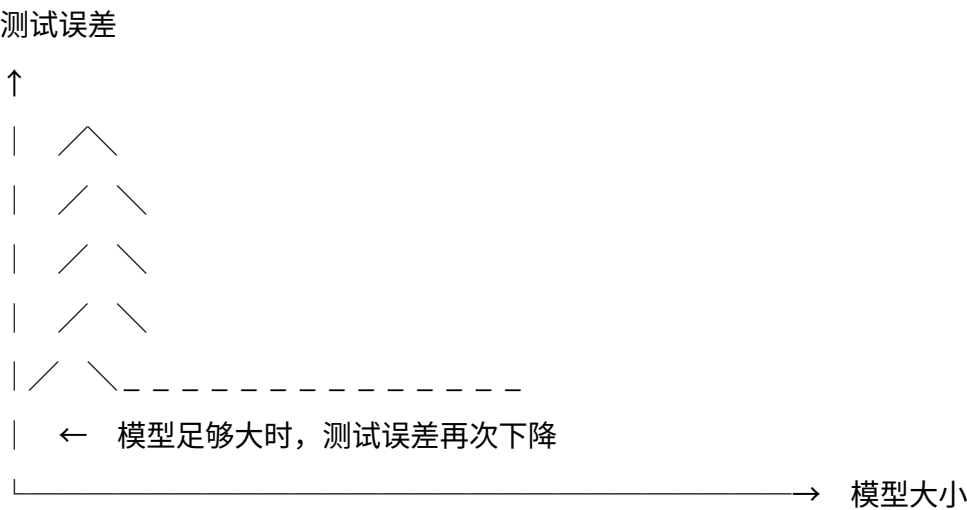
训练误差大，测试误差大 高偏差（欠拟合） 增加模型复杂度、添加特征、减少正则化

训练误差小，测试误差大 高方差（过拟合） 增加数据、增加正则化、降低模型复杂度、早停

训练误差小，测试误差也小 理想状态 —

深度学习中的特殊现象——双重下降（Double Descent）：

传统偏差 - 方差权衡认为模型越大越容易过拟合。但在深度学习中，当模型规模继续增大超过 " 过拟合峰 " 后，测试误差会再次下降——这就是双重下降现象。



欠拟合 过拟合峰 再次改善

这意味着：在深度学习中， " 模型太大 " 不一定是坏事——继续增大可能反而更好。

- - -

六、总结：数学知识体系

数学领域 在 A I 中的核心作用 典型应用

线性代数 提供数据表示和运算的基础

神经网络权重计算、P C A降维、特征分解、T r a n s f o r m e r 注意力、推荐系统

概率论 为决策和推理提供理论支持 朴素贝叶斯分类、贝叶斯网络、高斯分布建模、G A N噪声生成、A / B测试

最优化方法 A I 模型训练的核心驱动力 梯度下降、反向传播、A d a m / R M S P r o p / A d a g r a d

微积分 提供变化率和累积量的工具 梯度计算、反向传播（链式法则）、收敛性分析

统计学 * 提供从数据中推断的框架 假设检验、置信区间、偏差 - 方差权衡、A / B测试

线性代数为 A I 提供了数据表示和运算的基础，概率论为 A I 的决策和推理提供了理论支持，最优化方法则是 A I 模型训练的核心驱动力，微积分连接了这些工具（通过链式法则实现反向传播），统计学则提供了从数据中做出推断的严谨框架。这五者相互交织，共同构成了 A I 的数学基石。通过理解这些数学原理，我们能够更好地掌握 A I 技术的本质——不仅知道 " 怎么做 "，更能理解 " 为什么 "。